

III

GESTIÓN DEL ESTADO E INTERACTIVIDAD

En este capítulo se iniciará explorando los Stateful Widgets, que permitirán desarrollar secciones de la interfaz de usuario que se modifican dinámicamente en base a la interacción del usuario o datos en tiempo real. Luego, se descubrirá cómo los Inherited Widgets pueden optimizar la gestión del estado y el flujo de datos en toda la aplicación, promoviendo una estructura de código más organizada y eficiente.

El manejo adecuado de los gestos es fundamental para una experiencia de usuario fluida y gratificante. Se enseñará a implementar gestos comunes como tocar, deslizar y más, asegurando que las aplicaciones sean interactivas y placenteras de usar. Además, se abordarán los widgets de entrada, indispensables para capturar información del usuario.

Para aplicar estos conceptos, el ejercicio de este capítulo abordará el proceso de añadir interactividad a la aplicación de lista de tareas presentada anteriormente. Este paso reforzará el entendimiento de cómo utilizar los Stateful Widgets, gestionar entradas de usuario y emplear gestos para crear una experiencia de usuario rica y cautivadora.

3.1 Gestión del estado

Inicialmente es necesario entender qué es el estado y cómo afecta en una aplicación. En ese sentido, Arshad, p. (2021, p. 4) coadyuva en esta necesidad al definir que: “En términos simples, el estado de una aplicación es una condición o situación – una instancia o una instantánea que muestra la condición de tu aplicación en un determinado punto en el tiempo.” [Traducción propia].

La definición proporcionada sugiere que la función esencial de una interfaz de usuario (UI) es reflejar el estado, considerando al estado como una condición temporal que influye en la UI. Para clarificar este concepto, se puede recurrir al ejemplo de la descarga de una lista de artículos de internet. Inicialmente, es posible mostrar un widget de indicador de carga durante este proceso. Una vez que los datos han sido completamente cargados, el indicador de carga es reemplazado por la lista de artículos, evidenciando así una transición de un estado de carga a un estado cargado. La gestión manual de estos cambios de estado, sin seguir un método sistemático, puede llevar a la creación de código que resulta ser complejo, difícil de actualizar y de mantener. Por lo tanto, se hace imprescindible la adopción de un método estructurado para la gestión de estados, el cual debe especificar de manera clara cómo se deben monitorear los cambios de estado y cómo se deben comunicar los detalles sobre estos al resto de la aplicación.

Ahora bien, una vez definido el estado, es importante entender qué es la gestión del estado en Flutter.

En ese sentido, Arshad, p. (2021, p. 7) señala que:

La gestión de estados es simplemente una técnica, o múltiples técnicas, utilizadas para cuidar los cambios que ocurren en tu aplicación. Cuando tu aplicación se hace más grande, necesitas aplicar un enfoque de gestión de estado adecuado para poder rastrear cada cambio dentro de tu aplicación y hacer que la aplicación responda al usuario en consecuencia. [Traducción propia]

Este libro te mostrará diferentes técnicas de gestión de estados en Flutter, que van desde formas muy básicas, como `setState`, hasta enfoques avanzados, como `provider` (una técnica favorita de la comunidad ampliamente utilizada entre una variedad de aplicaciones). Para empezar, siempre es conveniente la manera básica, esto es lo que se presenta a continuación con los `Stateful Widgets`.

3.1.1 Stateful Widgets (Widgets con Estado)

En contraste con los widgets sin estado, los cuales reciben propiedades de su padre y que se mantienen sin alteración a lo largo de su duración, los widgets con estado han sido concebidos para modificar sus atributos de manera dinámica durante su ciclo de vida. Conforme a su definición, los widgets con estado son igualmente inmutables, pero incorporan una clase `State` que encapsula la representación del estado actual del widget. Este concepto se ilustra en el siguiente diagrama:

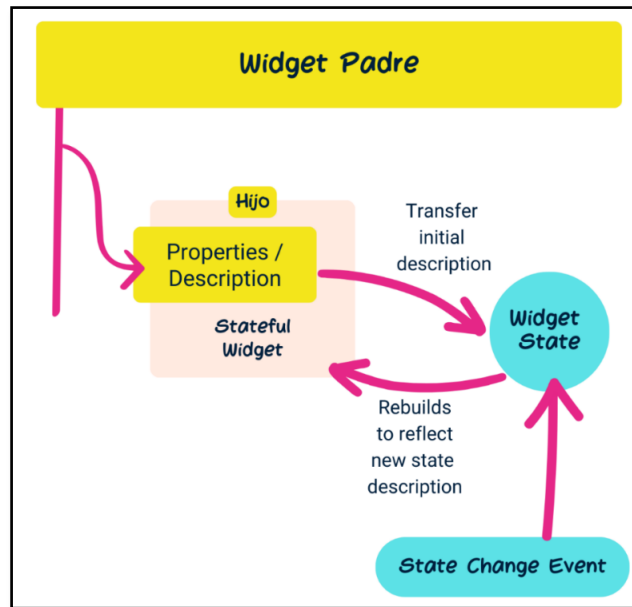


Figura III.1. *Stateful Widget adaptado de Flutter for beginners, por Bailey & Biessek, p. (2021, p. 117)*

En el diagrama anterior, se puede pensar en un widget como un elemento que crea y utiliza otro widget más pequeño dentro de él. De manera similar al ejemplo de un widget sin estado, este widget más grande le pasa información al widget más pequeño. Esta información que se pasa no puede cambiar una vez que se establece, es como si fuera *final* o fija. Sin embargo, en este caso, también hay un objeto llamado *State* que está relacionado con el widget más grande. Este objeto *State* tiene acceso a la información del widget, además, puede almacenar otra información que no necesariamente es fija, es decir, puede cambiar con el tiempo. Al mantener el estado del widget en un objeto *State* separado, el framework puede reconstruir el widget siempre que sea necesario sin perder su estado actual asociado. El elemento en el árbol de elementos mantiene una referencia al widget correspondiente y también al objeto *State* asociado con él. El objeto *State* notificará cuando el widget deba ser reconstruido y luego realizará la actualización correspondiente en el árbol de elementos.

A continuación, se presenta el código mínimo de un *StatefulWidget*:

```

class MyApp extends StatefulWidget {
  const MyApp({super.key});
  @override
  State<MyApp> createState() => _MyAppState();
}
class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    ...
  }
}
  
```

Lo primero a lo que se debe prestar atención es que la clase principal extiende *StatefulWidget*, identificando así que esto tendrá un objeto *State* asociado, los widgets con estado (Stateful widgets) deben sobre escribir el método *createState()* y devolver una instancia del objeto *State* asociado, este ejemplo, devuelve una instancia de *_MyAppState*. Un estado de widget válido es una clase que extiende la clase *State*, en este caso es la clase *_MyAppState*.

Ciclo de vida de un StatefulWidget.

Un widget con estado, conocido como *StatefulWidget*, atraviesa un ciclo de vida específico que comienza con su creación. Al instanciarse, Flutter primero llama al método *createState* para crear un objeto *State* que está vinculado al widget, el objeto *State* es donde se mantiene la lógica y el estado mutable del widget. El ciclo de vida del *State* comienza con el método *initState*, el cual se ejecuta una sola vez cuando el objeto *State* se inserta en el árbol de widgets, sirviendo para realizar inicializaciones, después de *initState*, se llama a *didChangeDependencies*, que se ejecuta cada vez que las dependencias del objeto *State* cambian, luego, el método *build* es llamado, este se encarga de construir la parte visual del widget, pudiendo ser llamado múltiples veces para reflejar los cambios en los datos o la configuración del widget. Si el widget cambia de configuración, *didUpdateWidget* se llama antes de *build*. Finalmente, cuando el widget se elimina del árbol de widgets, se llama a *dispose*, el cual es utilizado para liberar recursos y des suscribirse de servicios o streams. Este ciclo de vida permite una gestión eficiente y dinámica del estado dentro de la interfaz de usuario en Flutter.

El siguiente gráfico ilustra el proceso mencionado:

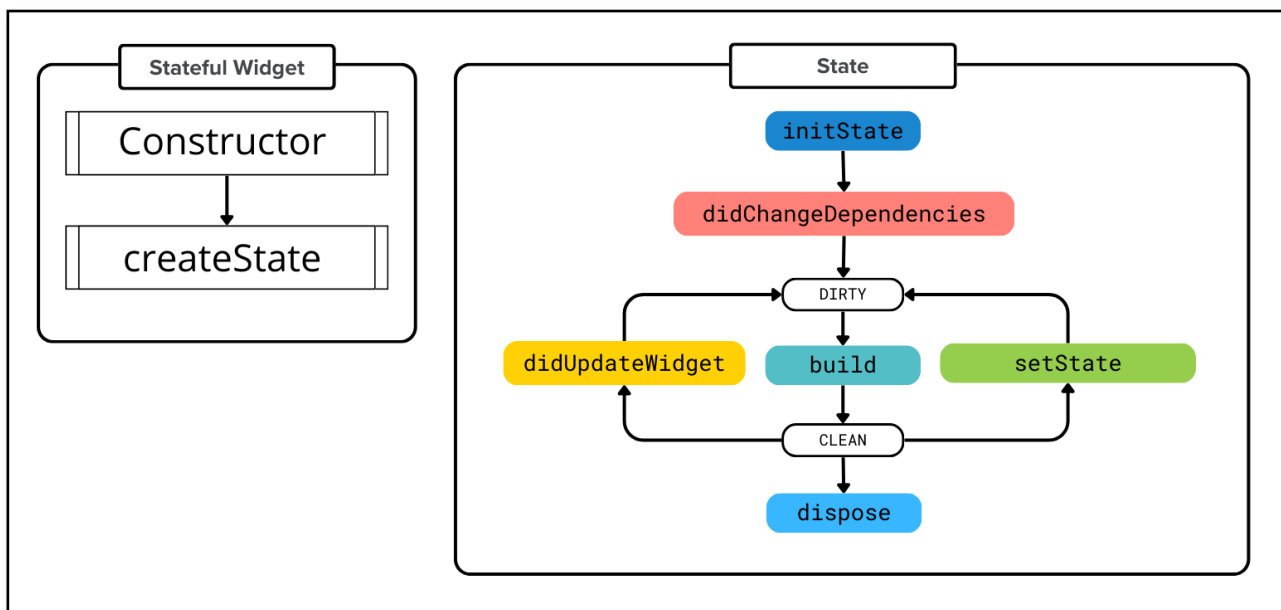


Figura III.2. Ciclo de Vida StatefulWidget

Se identifican algunas etapas clave en el ciclo de vida de un widget con estado que se explorarán en este punto, ya que su comprensión y manejo facilitarán la gestión de los datos. Estas etapas permiten entender cómo y cuándo se construye, actualiza y destruye un widget. Cada etapa tiene un propósito específico en la gestión del estado y la presentación del widget en la interfaz de usuario. Las etapas clave en el ciclo de vida de un *StatefulWidget* son:

1. **Creación del StatefulWidget.** Se llama al constructor del widget, esta es la primera etapa donde se inicializa el widget con los valores proporcionados.
2. **Creación del Estado.** Justo después de la creación del widget, Flutter llama al método *createState()* para crear una instancia de la clase *State* que está vinculada al widget, este es un paso decisivo porque es aquí donde se manejan los datos y la lógica del widget.
3. **Inicialización del Estado.** *initState()* se llama una vez que el objeto *State* ha sido creado, pero antes de que se construya el widget por primera vez, es el lugar para inicializar datos que dependen del contexto o realizar suscripciones a modelos o streams.
4. **Construcción del Widget.** *build()* es uno de los métodos más importantes y se llama justo después de *initState()* (la primera vez) y cada vez que es necesario reconstruir el widget, por ejemplo, cuando las dependencias cambian, aquí es donde se define la estructura del widget y cómo se debe visualizar.
5. **Actualización del Widget.** *didUpdateWidget()* se llama cuando el widget padre cambia y necesita reconstruirse, permite comparar la antigua instancia del widget con la nueva y reaccionar a los cambios.
6. **Cambio en el Árbol de Widgets.** *didChangeDependencies()*, este método se llama inmediatamente después de *initState()* la primera vez que se crea el objeto *State* y cuando las dependencias del widget cambian, siendo útil cuando el widget depende de modelos de datos heredados o de *InheritedWidgets*.
7. **Eliminación del Widget.** *dispose()* se llama justo antes de que el objeto *State* sea eliminado permanentemente y es el lugar para liberar recursos, cancelar suscripciones o timers, etc.
8. **Otros Métodos Importantes.** Aunque no son parte del ciclo de vida principal, hay otros métodos que pueden ser útiles:
 - *setState()* se llama para indicar que el estado del widget ha cambiado y que el método *build()* debe ser llamado de nuevo para reflejar esos cambios.
 - *deactivate()*, es llamado justo antes de que el objeto *State* se quite del árbol de widgets, puede ser útil para ciertas tareas de limpieza.

Estas etapas son fundamentales para entender cómo Flutter maneja los *StatefulWidget* y cómo interactúan con el resto del framework, conocer este ciclo de vida te permite crear aplicaciones más eficientes y responsivas.

Se desarrollará un ejemplo que permitirá una adecuada comprensión de un *Stateful Widget*. Este ejemplo permitirá manejar un contador que es posible incrementar desde un botón en la pantalla de la aplicación.

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MaterialApp(
    home: ContadorApp(),
  ));
}
class ContadorApp extends StatefulWidget {
  const ContadorApp({super.key});
  @override
  State<ContadorApp> createState() => _ContadorAppState();
}
class _ContadorAppState extends State<ContadorApp> {
  int _contador = 0;
  void _incrementarContador() {
    setState(() {
      _contador++;
    });
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ejemplo de Stateful Widget'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text('Número de veces que presionaste el boton:'),
            Text('$_contador'),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() {
          _incrementarContador();
        }),
        tooltip: 'Incrementar',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

3.1.2 Inherited Widgets (Widgets heredados)

Además de *StatelessWidget* y *StatefulWidget*, hay otro tipo de widget en el framework de Flutter, el *InheritedWidget*.

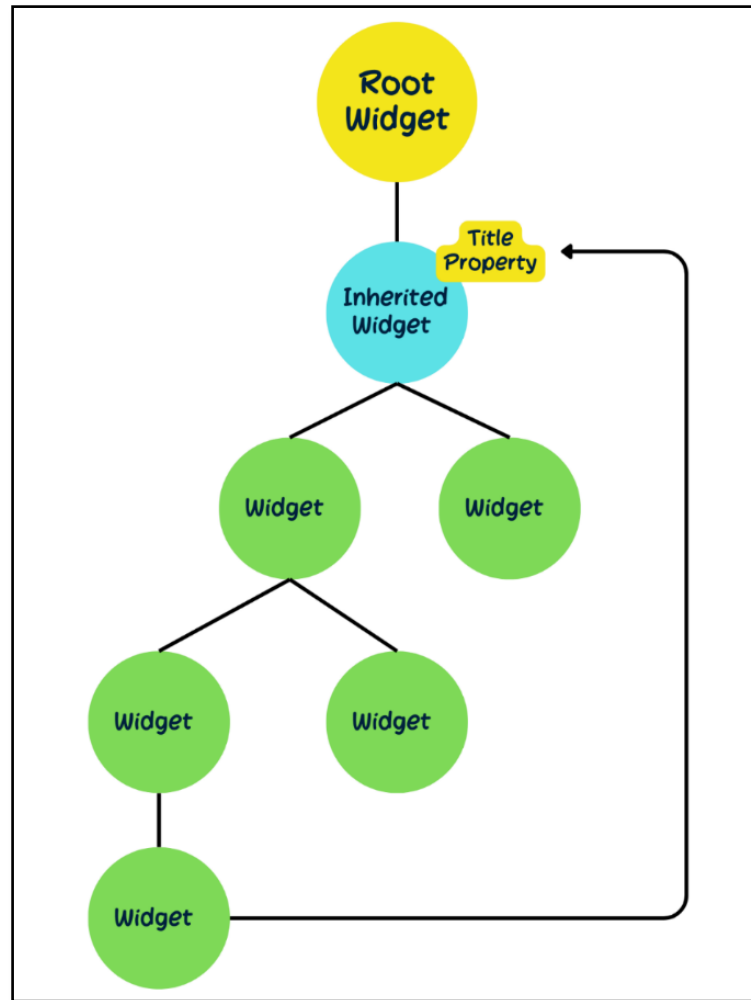


Figura III.3. *Inherited Widget* adaptado de *Flutter for beginners*, por Bailey & Biessek, p. (2021, p. 124)

Un widget de este tipo es relevante cuando un widget puede requerir acceso a datos más arriba en el árbol de widgets. En tales escenarios, una solución es replicar la información hasta el widget interesado, pasándola a través de todos los widgets intermedios.

Si se considera un escenario en el que un widget situado más abajo en el árbol necesita acceder a la propiedad *title* desde el widget raíz, en un entorno donde solo existen widgets con estado (*stateful*) y sin estado (*stateless*), sería necesario pasar la propiedad *title* a cada widget hijo a través de los constructores, esto permitiría que el widget hijo, a su vez, pudiera pasar la propiedad *title* a sus widgets hijos. Esta práctica puede resultar en una gran cantidad de código repetitivo, propiciar errores si uno de los widgets no se codifica correctamente, y complicarse significativamente si se decide que el widget hijo necesita otra propiedad, lo que implicaría que todos los widgets intermedios deben actualizarse.

Para abordar este problema, Flutter proporciona la clase *InheritedWidget*, al agregar un *InheritedWidget* al árbol, cualquier widget debajo de él puede acceder a los datos que expone utilizando el método de (*InheritedWidget*) de la clase *BuildContext*, este método recibe un tipo de *InheritedWidget* como parámetro y utiliza el árbol para encontrar el primer widget ancestral del tipo solicitado. Esta es una especie de widget auxiliar que ayuda a propagar información hacia abajo en el árbol, como se muestra en el siguiente diagrama:

Un *InheritedWidget* se define creando una clase que extienda *InheritedWidget*. Esta clase debe sobrescribir el método *updateShouldNotify* para determinar cuándo se deben notificar a los widgets hijos sobre los cambios. La estructura típica de un *InheritedWidget* incluye:

- Variables para almacenar los datos que se desean compartir.
- Un constructor para inicializar el widget y sus datos.
- El método *of*, que permite a los widgets hijos acceder a los datos.

Recuérdese el ejemplo anterior, en el que se desarrolló un contador empleando *StatefulWidget*, porque ahora se trabajará en la misma funcionalidad pero empleando *InheritedWidget*.

```
class ContadorProvider extends InheritedWidget {  
  final int contador;  
  final Widget child;  
  const ContadorProvider({Key? key, required this.contador, required this.child})  
    : super(key: key, child: child);  
  static ContadorProvider? of(BuildContext context) {  
    return context.dependOnInheritedWidgetOfExactType<ContadorProvider>();  
  }  
  @override  
  bool updateShouldNotify(ContadorProvider oldWidget) {  
    return contador != oldWidget.contador;  
  }  
}
```

ContadorProvider es una clase que hereda de *InheritedWidget*, esta clase almacena el valor del contador y al tratarse de un *InheritedWidget* permitirá a otros widgets acceder a este valor. La función *of(BuildContext context)* se utiliza para acceder a la instancia más cercana de este tipo de widget en el árbol de widgets y *updateShouldNotify* determina si los widgets que dependen de este deben reconstruirse cuando el contador cambia.

Una vez que se tiene el *InheritedWidget*, se debe crear el *StatefulWidget* que utiliza el *InheritedWidget*.

```
class ContadorApp extends StatefulWidget {
  const ContadorApp({super.key});
  @override
  State<ContadorApp> createState() => _ContadorAppState();
}
class _ContadorAppState extends State<ContadorApp> {
  int _contador = 0;
  void _incrementarContador() {
    setState(() {
      _contador++;
    });
  }
  @override
  Widget build(BuildContext context) {
    return ContadorProvider(
      contador: _contador,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('Ejemplo Inherited Widget'),
        ),
        body: const Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Text('Número de veces que presionaste el boton:'),
              ContadorConsumidor(),
            ],
          ),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: _incrementarContador,
          tooltip: 'Incrementar',
          child: const Icon(Icons.add),
        ),
      ),
    );
  }
}
```

ContadorApp es un *StatefulWidget* que gestiona el estado del contador, este widget incrementa el valor del contador y utiliza *ContadorProvider* para hacer que el valor esté disponible para otros widgets en la jerarquía.

Seguramente se notó un objeto *ContadorConsumidor()*, este corresponde a la siguiente clase:

```
class ContadorConsumidor extends StatelessWidget {
  const ContadorConsumidor({super.key});

  @override
  Widget build(BuildContext context) {
    final contadorProvider = ContadorProvider.of(context);

    return Text(
      '${contadorProvider?.contador ?? 0}'
    );
  }
}
```

ContadorConsumidor es un *StatelessWidget* que usa *ContadorProvider.of(context)* para acceder al valor actual del contador, este widget se reconstruye automáticamente cuando el valor del contador cambia, gracias a la implementación de *updateShouldNotify* en *ContadorProvider*.

Finalmente, el método *main()* inicia la aplicación con *MaterialApp*, estableciendo *ContadorApp* como el widget inicial.

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MaterialApp(
    home: ContadorApp(),
  ));
}
```

El código anterior proporciona un buen ejemplo de cómo estructurar una aplicación Flutter utilizando *InheritedWidget* para la gestión del estado, manteniendo un código limpio y una estructura de proyecto clara.

3.2 Manejo de gestos del usuario

Según Windmill, p. (2020, p. 130) “Los gestos son cualquier tipo de evento de interacción: toques, arrastres, desplazamientos y más.” [Traducción propia].

En ese sentido, una aplicación móvil estaría muy limitada sin la posibilidad de reconocer la interacción del usuario, a esa interacción se la conoce como gestos, a continuación, se pasará a estudiarlos de manera detallada.

3.2.1 Gestos

Los eventos de pantalla en el sistema de gestos de Flutter se dividen en dos capas:

- La capa de punteros (Pointers layer)
- La capa de gestos (Gestures layer).

La capa de punteros es la capa más baja y básica en el sistema de gestos de Flutter. En esta capa, se maneja la información cruda sobre cómo un puntero (como un toque, un ratón o un lápiz óptico) está interactuando con la pantalla, esta información incluye la ubicación y el movimiento del puntero, es la primera capa donde Flutter comienza a manejar la entrada de la pantalla.

La capa de gestos toma múltiples acciones del puntero y trata de asignarles un significado como una acción del usuario, estas acciones semánticas incluyen gestos como toques, arrastres o gestos de escala, esta capa es la forma más típica de manejar la entrada del usuario en aplicaciones Flutter. En lugar de lidiar con la información cruda de la capa de punteros, la capa de gestos interpreta estas entradas en acciones más significativas y útiles para la aplicación.

En resumen, la capa de punteros y la capa de gestos trabajan juntas para proporcionar un manejo completo y flexible de la entrada del usuario en Flutter, desde la detección de la interacción básica con la pantalla hasta la interpretación de gestos complejos.

Aunque es posible, no siempre es práctico manejar eventos de puntero. En su lugar, los eventos pueden ser manejados en la segunda capa del sistema de gestos. Los gestos son reconocidos a partir de múltiples eventos de puntero e incluso de múltiples punteros individuales (multitouch), estos son algunos de los tipos de gestos más comunes que se pueden manejar en esta capa:

- **Tap:** Un toque simple en la pantalla del dispositivo.
- **Double-tap:** Un doble toque rápido en la misma ubicación de la pantalla.
- **Press y Long-press:** Un toque en la pantalla que se mantiene por un período de tiempo más largo antes de soltarlo, similar a un tap pero con una duración más larga.
- **Drag:** Un toque que comienza con un contacto en algún lugar de la pantalla y luego se mueve, terminando el contacto en otra ubicación.
- **Pan:** Similar a los eventos de arrastre (drag), pero en Flutter cubren tanto el arrastre horizontal como el vertical.
- **Scale:** Se utilizan dos punteros para un movimiento de arrastre que emplea un gesto de escala, similar a un gesto de zoom.

Flutter proporciona el widget *GestureDetector*, que contiene devoluciones de llamada para todos los eventos mencionados anteriormente. Debes usarlos según la interacción que desee permitir. Echemos un vistazo a algunos ejemplos de *GestureDetector*.

3.2.2 El Widget GestureDetector

En el siguiente ejemplo, se presenta un widget con estado denominado *TapExample* y su clase de estado asociada, *_TapExampleState*. Esto sigue la estructura del widget con estado que se observó en el capítulo anterior. Se necesita tener un estado porque se requiere que el widget reaccione a las interacciones del usuario, lo cual implicará cambiar el valor de una variable y luego redibujar el widget.

Inicialmente se presenta el código y luego se procederá a analizar brevemente cómo funciona.

```
class TapExample extends StatefulWidget {
  const TapExample({super.key});
  @override
  State<TapExample> createState() => _TapExampleState();
}
class _TapExampleState extends State<TapExample> {
  int _counter = 0;
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () {
        setState(() {
          _counter++;
        });
      },
      child: Container(
        width: 100,
        height: 100,
        alignment: Alignment.center,
        color: Colors.green,
        child: Text(
          "Tap count: $_counter",
        ),
      ),
    );
  }
}
```

TapExample es un *StatefulWidget*, lo que significa que su estado puede cambiar durante el tiempo de vida del widget, este widget sobrescribe el método *createState()* que devuelve una instancia de *_TapExampleState*, que es donde se manejará la lógica y el estado del widget.

En la Clase *_TapExampleState* se tiene *_counter* que es una variable que se utiliza para mantener el conteo de los toques, esta variable se inicializa en 0. El método *build* retorna un widget *GestureDetector* y este a su vez envuelve a un hijo *Container*, eso significa que, *GestureDetector* detecta gestos de toque (tap) sobre el *Container*. Cuando se detecta un toque, el método *onTap* se activa, incrementando *_counter* mediante *setState()*, esto causa que el widget se reconstruya con el valor actualizado de *_counter*.

El Widget *Container* define las propiedades visuales como dimensiones (*width*, *height*) y color (*color*) y además, dentro del *Container*, hay un widget *Text* que muestra el número actual de toques.

Es muy importante entender que, cada vez que se toca el *Container*, *setState()* se llama y actualiza el valor de *_counter*. Esto desencadena una reconstrucción del widget, actualizando la interfaz con el nuevo valor de *_counter*.

Este código es un ejemplo básico pero fundamental para entender cómo se manejan los gestos y cómo se actualiza dinámicamente la interfaz de usuario en respuesta a las interacciones del usuario en Flutter, es una práctica común para manejar estados locales en widgets interactivos.

El uso de *GestureDetector* para capturar gestos es notablemente sencillo, por lo que se procederá a explorar los gestos disponibles.

Tap

Un gesto de toque implica que un puntero toque la pantalla en una ubicación y luego termine el contacto en una ubicación similar, sin alejarse de esa ubicación.

En el ejemplo anterior, se utilizó específicamente el parámetro constructor *onTap* de *GestureDetector*. Se analizará más de cerca la sección clave del código:

```
onTap: () {  
  setState(() {  
    _counter++;  
  });  
},
```

El parámetro *onTap* toma una función como argumento y se colocó una función en línea que incrementa la variable *_counter* dentro del método *setState()*.

Si deseas escuchar gestos más detallados alrededor de los toques, también hay otros parámetros de constructor en *GestureDetector* que se pueden usar, por ejemplo:

- *onTapDown*: Un puntero que puede estar a punto de tocar ha entrado en contacto con la pantalla.
- *onTapUp*: Un puntero que ha tocado la pantalla ha terminado de tocar la pantalla.
- *onTapCancel*: Un puntero que activó *onTapDown* (por lo que la capa de gestos pensó que había una posibilidad de que el gesto completo fuera un toque) en realidad no hizo un toque.

Double-tap

Un doble toque tiene restricciones muy similares a un toque, pero agrega la estipulación de que debe haber dos toques en rápida sucesión.

El código siguiente, se verá muy similar al ejemplo de toque ya que lo único que cambia es el parámetro *onDoubleTap*:

```
onDoubleTap: () {  
  setState(() {  
    _counter++;  
  });  
},
```

La única diferencia respecto al ejemplo anterior es la utilización del parámetro constructor *onDoubleTap*. Se ha pasado la misma función en línea como argumento y dicha función se llamará cada vez que se realicen dobles toques en la misma ubicación en la pantalla.

Si lo que se desea es escuchar gestos más detallados alrededor de los dobles toques, también hay otros parámetros de constructor en *GestureDetector* que se pueden usar:

- *onDoubleTapDown*: Un puntero que puede estar a punto de realizar un doble toque ha entrado en contacto con la pantalla.
- *onDoubleTapUp*: Un puntero que ha realizado un doble toque en la pantalla ha terminado de tocar la pantalla.
- *onDoubleTapCancel*: Un puntero que activó *onDoubleTapDown* (por lo que la capa de gestos pensó que había una posibilidad de que el gesto completo fuera un doble toque) en realidad no realizó un doble toque.

Debe considerarse que algunos de los gestos más detallados o parciales pueden activarse antes de que se resuelva el gesto completo y, por lo tanto, no coincidir con el gesto correcto. Por ejemplo, *onTapDown* se activaría en el primer toque de un doble toque, pero en última instancia, el gesto de toque no se completaría porque el usuario realizó un doble toque en lugar de un toque único. Por lo tanto, se recomienda precaución al utilizar estos gestos parciales.

Press and hold

Un “long press” en la pantalla del dispositivo es similar a un toque, pero implica mantener el contacto con la pantalla durante un período de tiempo más largo antes de soltarlo y sin alejarse de la ubicación.

El código es muy similar a los ejemplos anteriores:

```
onLongPress: () {  
  setState(() {  
    _counter++;  
  });  
},
```

La única diferencia respecto a los elementos anteriores es la propiedad asignada, *onLongPress*, que se llamará cada vez que se realice un toque y se mantenga presionado durante algún tiempo antes de ser liberado de la pantalla.

Si deseas escuchar gestos más detallados alrededor del toque prolongado, también hay otros parámetros de constructor en *GestureDetector* que puedes usar:

- *onLongPressStart*: Se ha reconocido un puntero en contacto con la pantalla como realizando un toque prolongado.
- *onLongPressEnd/onLongPressUp*: Un puntero que ha realizado un toque prolongado en la pantalla ha terminado de tocar la pantalla.
- *onLongPressMoveUpdate*: Un puntero que activó *onLongPressStart* ha sido arrastrado.

Es posible utilizar muchos de los diferentes gestos en el mismo *GestureDetector*, ya que él decidirá qué gesto y si alguno es aplicable al toque, llamará a la función suministrada para ese gesto.

Drag, pan y scale

Los gestos *drag*, *pan* y *scale* son similares entre sí y se deberá decidir cuál usar en cada situación, ya que no se pueden usar todos juntos en el mismo widget *GestureDetector*. Se retrasará la explicación de los mismos para una versión posterior del libro; sin embargo, si se requiere emplearlos, siempre es posible recurrir a la documentación oficial, esta vez en la sección de gestos³³.

3.2.3 Gestos en Widgets de Material

Si bien *GestureDetector* es un widget muy útil, la mayoría de las veces no será necesario utilizarlo, ya que los widgets integrados ya tienen la gestión de gestos incorporado, los widgets de Material Design tienen muchos gestos abstraídos como un parámetro de constructor utilizando internamente el widget *GestureDetector* en su código.

Flutter proporciona una serie de widgets que ya tienen incorporados gestos comunes, por ejemplo:

- **Botones (*ElevatedButton*, *OutlinedButton*, etc.):** Manejan automáticamente los gestos de tap.
- *ListTiles*: Responden a taps, mostrando un efecto visual al ser presionados.
- *Sliders* y *Switches*: Responden a gestos de drag para cambiar sus valores.
- *Dismissible*: Un widget que se puede deslizar para realizar acciones como eliminar o archivar.

Por otro lado, si se requiere agregar interactividad a los widgets que no tienen gestos incorporados, es posible usar *GestureDetector* como se vio en el anterior punto. Por ejemplo, se puede usar *onTap*, *onDoubleTap*, *onLongPress* y más, para ejecutar cualquier código cuando se detecte ese gesto, sin embargo, se debe recordar que, al implementar gestos, es importante considerar la usabilidad y accesibilidad:

- Los gestos deben ser intuitivos y consistentes con las expectativas del usuario.
- Debe haber retroalimentación visual para indicar que un gesto ha sido reconocido.

³³ <https://docs.flutter.dev/ui/interactivity/gestures>

- Considera la accesibilidad, asegurándote de que los gestos sean accesibles para usuarios con diferentes capacidades.

En resumen, los gestos en los widgets de Material son una combinación de funcionalidades predefinidas en widgets específicos y la capacidad de personalizar la interactividad a través de widgets como *GestureDetector*. Esto permite crear interfaces ricas e interactivas que son una parte esencial de la experiencia de usuario en aplicaciones móviles modernas.

Para ilustrar lo descrito anteriormente, se presentará el código de cómo crear un switch personalizado en Flutter usando *StatefulWidget*. Este widget, denominado *MySwitch*, permite al usuario alternar entre dos estados: ON y OFF. A continuación, se examinará el código y se desglosará el funcionamiento de este widget:

```
class MySwitch extends StatefulWidget {
  const MySwitch({super.key});
  @override
  State<MySwitch> createState() => _MySwitchState();
}
class _MySwitchState extends State<MySwitch> {
  bool isSwitched = false;
  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Text(
          'Switch is ${isSwitched ? 'ON' : 'OFF'}',
          style: const TextStyle(fontSize: 20),
        ),
        Switch(
          value: isSwitched,
          onChanged: (value) {
            setState(() {
              isSwitched = value;
            });
          },
          activeTrackColor: Colors.lightGreenAccent,
          activeColor: Colors.green,
        ),
      ],
    );
  }
}
```

Se hace imprescindible realizar una explicación del código presentado, comenzando con la definición de la clase *MySwitch* que es un *StatefulWidget*, lo que implica que posee un estado susceptible de cambiar a lo largo del tiempo, el método *createState()* devuelve una instancia de *_MySwitchState*, que es donde se gestiona el estado del widget.

El estado del widget es manejado por la clase *_MySwitchState* y contempla una variable y un método miembros de la clase, su utilidad se explica a continuación. La variable *isSwitched* mantiene el estado actual del switch, se inicializa con *false*, indicando que el switch está en estado OFF inicialmente.

El método *build()* se dedica a la construcción de la UI para ello emplea un *Column* para organizar verticalmente los widgets, se muestra un *Text* que refleja el estado actual del switch (ON u OFF), dependiendo del valor de *isSwitched*, el widget “Switch” muestra el interruptor que el usuario puede tocar para cambiar su estado.

Al examinar más detalladamente el widget Switch, se evidencia que la propiedad *value* refleja el estado actual del *Switch*, vinculado a la variable *isSwitched*. El evento *onChanged* es un callback que se invoca cada vez que el usuario modifica el estado del *Switch*, dentro de este callback, se llama a *setState()*, lo que provoca una reconstrucción del widget con el nuevo estado, ya que se actualiza el valor de *isSwitched* a *value*, correspondiente al nuevo estado del *Switch*. Las propiedades *activeTrackColor* y *activeColor* permiten personalizar los colores del switch dependiendo de su estado.

Este ejemplo demuestra cómo manejar estados y cómo los widgets interactivos pueden cambiar su apariencia y función basándose en las acciones del usuario, el uso de *setState* en este contexto es fundamental para actualizar la UI de manera reactiva.

3.3 Widgets de entrada

La capacidad de la aplicación para gestionar gestos es un buen punto de partida para la interacción con el usuario, pero para muchas aplicaciones, también se requiere una forma de obtener otros tipos de entrada por parte del usuario que permitan agregar contenido personalizado a las aplicaciones.

La plataforma Flutter, ofrece un conjunto robusto de widgets para la creación de interfaces de usuario interactivas y dinámicas en aplicaciones móviles, *TextField* por ejemplo es un componente fundamental en el desarrollo de aplicaciones con Flutter, su correcta utilización, puede significar una mejora sustancial en la experiencia del usuario y la eficacia en la gestión de datos dentro de las aplicaciones móviles.

3.3.1 Manejo de la entrada con TextField

El *TextField* es un widget específicamente diseñado para la entrada de texto, este componente es ampliamente utilizado en la captura de información textual sencilla como nombres, direcciones y comentarios. Su implementación es notablemente directa, ofreciendo a los desarrolladores un control eficiente sobre la entrada de texto a través del *TextEditingController* o el método *onChanged*.

El método *onChanged* se puede usar para escuchar los cambios de la entrada. Sin embargo, otra manera de escuchar cambios es utilizando un controlador, esto permite manipular y obtener el contenido textual, además de facilitar la implementación de lógicas para el seguimiento y respuesta a cambios en el texto.

El *TextField* también proporciona una amplia gama de opciones de personalización visual, permitiendo adaptar la apariencia del campo de texto a las necesidades estéticas y funcionales de la aplicación. En ese contexto y con el objetivo de prevenir cualquier posible confusión en la implementación de un *TextField*, se proporcionará una descripción detallada de la estructura completa para su aplicación en un ejemplo íntegro.

El ejemplo comienza con la función *main()*, que ejecuta la aplicación mediante la función *runApp()*, tomando como argumento el widget raíz, en este caso *MyApp*. La clase *MyApp* es un *StatefulWidget*, lo que indica que su estado puede cambiar durante el tiempo de vida de la aplicación, la naturaleza dinámica de *MyApp* se gestiona a través de la clase *_MyAppState*, que extiende *State<MyApp>*.

Se observa lo mencionado en el siguiente código:

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatefulWidget {
  const MyApp({super.key});
  @override
  State<MyApp> createState() => _MyAppState();
}
```

Un aspecto crítico en la gestión de un *TextField* es el control de su estado, lo cual se realiza mediante la instancia *_controller* del tipo *TextEditingController*, este controlador vincula el estado del texto dentro del *TextField* con la lógica de la aplicación.

Por lo tanto, el siguiente código incluye el controlador mencionado dentro la clase *_MyAppState*.

```
class _MyAppState extends State<MyApp> {
  final TextEditingController _controller = TextEditingController();
  String _textValue = '';
  //...
}
```

Dentro la misma clase, la función *initState()* inicializa el estado del widget. Aquí, se añade un escuchador al *_controller*, que desencadena una llamada a *setState()* cada vez que el usuario modifica el texto en el *TextField*. Esta práctica es coherente con el paradigma de programación reactiva, donde los cambios en la interfaz de usuario se reflejan en tiempo real en respuesta a las interacciones del usuario.

Lo mencionado se refleja en el siguiente código:

```
@override
void initState() {
  super.initState();
  _controller.addListener(() {
    setState(() {
      _textValue = _controller.text;
    });
  });
}
```

Aún dentro la misma clase, el método *build()* construye la interfaz de usuario, utilizando *Scaffold* para estructurar la aplicación. El *TextField* se encapsula dentro de un widget *Padding*, asegurando que haya un espacio adecuado alrededor del campo de texto, lo que mejora la estética y la accesibilidad, el atributo *controller* del *TextField* se enlaza con *_controller*, estableciendo una conexión directa entre la entrada del usuario y el mecanismo de control del estado.

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Ejemplo TextField'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: TextField(
                controller: _controller,
                decoration: const InputDecoration(labelText: 'Ingrese texto'),
              ),
            ),
            const SizedBox(height: 20),
            Text('Texto ingresado: $_textValue'),
          ],
        ),
      ),
    ),
  );
}
```

Para finalizar la clase, es imprescindible liberar los recursos asociados al controlador cuando el estado del widget se destruye, la función `dispose()` se encarga de esto, llamando a `_controller.dispose()` lo que previene las fugas de memoria.

```
@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
```

En resumen, este ejemplo revela cómo Flutter facilita una gestión eficiente y efectiva de los campos de texto a través de `TextEditingController` y cómo los patrones de diseño reactivos se integran en la arquitectura de las aplicaciones Flutter, la capacidad de Flutter para actualizar dinámicamente la interfaz de usuario en respuesta a las interacciones del usuario demuestra su eficacia en la creación de aplicaciones móviles interactivas y centradas en el usuario.

3.3.2 El Widget de Form

Como señala Freitas, p. (2019, p. 75) “Al igual que con HTML, Flutter tiene el concepto de un formulario, y como podrías haber adivinado, los formularios en Flutter también pueden enviarse. Enviar un formulario es lo que desencadena la acción de guardar.” [Traducción propia].

En ese sentido, el widget `Form` en Flutter es un contenedor que se utiliza para agrupar uno o más widgets de campo de formulario, como `TextFormField`, `DropDownButtonFormField`, entre otros. Es un elemento esencial en el desarrollo de interfaces de usuario en Flutter para la creación de formularios interactivos, este widget no solo agrupa los campos de entrada, sino que también proporciona la funcionalidad para validar y manejar los datos ingresados por los usuarios.

Algunas de sus funcionalidades clave son:

- **Agrupación de campos de formulario:** Permite organizar múltiples widgets de campo de formulario bajo un único paraguas, facilitando su manejo y validación.
- **Gestión del estado del formulario:** A través del uso de un `GlobalKey<FormState>`, `Form` permite controlar el estado de los campos de formulario, incluyendo la obtención de los valores de entrada y la realización de validaciones.
- **Validación de datos:** El widget `Form` puede validar automáticamente los datos ingresados en cada campo de formulario cuando el usuario intenta enviar el formulario, cada campo individual puede tener su propia lógica de validación y el formulario en su conjunto se considera válido solo si todos los campos pasan sus respectivas validaciones.

En el desarrollo de aplicaciones con Flutter, el formulario es una parte integral de la interfaz de usuario para la recolección de datos, el widget *Form* juega un papel importante en este proceso, proporcionando un marco estructurado y eficiente para:

- **Recopilar información del usuario:** Desde datos simples como nombre y correo electrónico hasta información más compleja como preferencias y comentarios.
- **Validar la entrada del usuario:** Asegurando que los datos proporcionados cumplan con los criterios específicos necesarios para el procesamiento o almacenamiento.
- **Facilitar la interacción del usuario:** Ofreciendo una experiencia de usuario coherente y accesible para formularios de cualquier complejidad.
- **Integrar con backend y servicios de datos:** Permitiendo una fácil recolección y transmisión de datos hacia y desde sistemas backend, APIs, o bases de datos.

El widget *Form* es, por lo tanto, un componente fundamental en el kit de herramientas de Flutter para el desarrollo de aplicaciones móviles, proporciona una estructura robusta y flexible para la creación de formularios, facilitando la recopilación, validación y manejo de datos de entrada de manera eficiente y efectiva.

La correcta implementación y uso del widget *Form* son esenciales para garantizar una experiencia de usuario fluida y profesional en cualquier aplicación Flutter que requiera la recopilación de datos del usuario.

Conocidas las características teóricas principales del Widget, se procede a su aplicación práctica. El manejo del estado de un formulario es un aspecto fundamental en el desarrollo de aplicaciones con Flutter, implica controlar los valores de los campos de entrada, realizar la validación de los datos y responder a los cambios de estado de forma efectiva. A continuación, se presentan prácticas recomendadas para el manejo eficiente del estado del formulario en Flutter.

Uso de GlobalKey<FormState>

Para gestionar el estado de un formulario, Flutter proporciona el *GlobalKey<FormState>*. Este *key* es esencial para interactuar con el formulario, como, por ejemplo, para realizar validaciones o para resetear el formulario.

```
final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
```

Se asigna este *key* al widget *Form* para mantener una referencia al estado del formulario.

Limpieza y Reseteo del Formulario

Para limpiar o resetear un formulario, puedes usar el método *reset()* del *FormState*, esto restablecerá todos los campos del formulario a sus valores iniciales.

```
_formKey.currentState.reset();
```

Gestión del Estado en Formularios Dinámicos

En formularios complejos o dinámicos, donde los campos pueden cambiar en función de las acciones del usuario, considera el uso de un estado más avanzado o patrones de gestión de estado como *Provider*, *Bloc* o *Riverpod*.

Separación de la Lógica del Estado

Para mantener el código mantenible y escalable, separa la lógica del estado del formulario de la UI. Utiliza modelos, clases de controlador o lógica de negocio separada para gestionar el estado del formulario.

3.3.3 Estructura básica de un formulario

La creación de un formulario en Flutter implica una comprensión de su estructura jerárquica y la importancia del contexto dentro de esta jerarquía, un formulario en Flutter se construye utilizando una combinación de varios widgets, cada uno desempeñando un papel específico en la recopilación y manejo de datos de entrada.

Jerarquía de Widgets en un Formulario

El Widget *Form* es el contenedor principal de todos los campos de formulario, se utiliza para agrupar varios widgets de entrada de datos, proporcionando un contexto común para todos ellos.

A continuación, puedes observar cómo luce el código de un widget *Form*:

```
Form(  
  key: _formKey, // Clave global para identificar el formulario  
  child: Column(  
    children: <Widget>[ // Los campos de formulario van aquí ],  
  ),  
)
```

Los campos de formulario (*TextFormField*, *DropDownButtonFormField*, etc.) son los widgets individuales que recogen la entrada del usuario, cada uno de estos widgets puede tener su propia lógica de validación. Para tener una referencia, a continuación, puedes observar su sintaxis:

```
TextFormField(  
  validator: (value) {  
    if (value.isEmpty) { return 'Este campo no puede estar vacío'; }  
    return null;  
  },  
)
```

Los botones o widgets de acción como por ejemplo *ElevatedButton* o widgets similares que el usuario puede tocar para enviar los datos del formulario o realizar otras acciones, pueden tener el siguiente aspecto.

```
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState.validate()) { // Procesar datos del formulario }
  },
  child: Text('Enviar'),
)
```

A continuación, se presenta el ejemplo completo para proporcionar mayor claridad sobre lo mencionado. En este ejemplo, el *Form* agrupa dos *TextFormField* y un *ElevatedButton*.

```
import 'package:flutter/material.dart';
void main() { runApp(const MyApp()); }
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Form Example',
      home: Scaffold(
        appBar: AppBar(title: const Text('Formulario de Inicio de Sesión')),
        body: const LoginForm(),
      ),
    );
  }
}
class LoginForm extends StatefulWidget {
  const LoginForm({super.key});
  @override
  State<LoginForm> createState() => _LoginFormState();
}
class _LoginFormState extends State<LoginForm> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  String _username = ''; String _password = '';
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(20.0),
      child: Form(
        key: _formKey,
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: <Widget>[
            TextFormField(
```

```

        decoration: const InputDecoration(
          labelText: 'Usuario',
          border: OutlineInputBorder(),
        ),
        validator: (value) {
          if (value!.isEmpty) {
            return 'Ingrese su usuario';
          }
          _username = value;
          return null;
        },
      ),
      const SizedBox(height: 20),
      TextFormField(
        decoration: const InputDecoration(
          labelText: 'Contraseña',
          border: OutlineInputBorder(),
        ),
        obscureText: true,
        validator: (value) {
          if (value!.isEmpty) {
            return 'Ingrese su contraseña';
          }
          _password = value;
          return null;
        },
      ),
      const SizedBox(height: 20),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Usuario: $_username, Contraseña:
$_password')),
            );
          }
        },
        child: const Text('Iniciar Sesión'),
      ),
    ],
  ),
),
);
}
}

```

La estructura de un formulario en Flutter es un sistema jerárquico que combina varios widgets dentro de un contexto común, comprender esta estructura y la importancia del contexto es esencial para construir formularios eficientes y efectivos en aplicaciones Flutter.

3.3.4 Técnicas para implementar validaciones y mostrar mensajes de error

La validación de datos de entrada es un aspecto decisivo en el desarrollo de formularios, proporciona una manera de asegurar que los datos ingresados por los usuarios cumplan con los criterios específicos antes de ser procesados o almacenados. A continuación, se describen algunas técnicas efectivas para implementar estas validaciones y mostrar mensajes de error adecuados.

Uso del Método `validator` en `TextFormField`

Cada `TextFormField` en Flutter puede tener una función `validator` asociada, esta función se ejecuta automáticamente cuando se llama al método `validate` del `FormState`.

```
TextFormField(  
  validator: (value) {  
    if (value.isEmpty) { return 'Este campo no puede estar vacío'; }  
    return null; // Retorna null si el dato es válido  
  },  
)
```

Personalización de Mensajes de Error

Los mensajes de error deben ser claros y útiles, indicando al usuario exactamente qué necesita ser corregido.

```
String _validateEmail(String value) {  
  if (value.isEmpty) { return 'Ingresa un correo electrónico'; }  
  else if (!value.contains('@')) { return 'Ingresa un correo electrónico válido'; }  
  return null;  
}
```

Validación en Tiempo Real

La validación en tiempo real mejora la experiencia del usuario al proporcionar retroalimentación instantánea. Esto se puede lograr utilizando el parámetro `autovalidateMode` en `Form`.

```
Form(  
  autovalidateMode: AutovalidateMode.onUserInteraction,  
  // Valida cada campo después de la interacción del usuario  
)
```

Agrupación de Múltiples Validaciones

Para campos que requieren múltiples criterios de validación, es posible combinar varias funciones de validación.

```
String _validatePassword(String value) {  
  if (value.isEmpty) { return 'La contraseña no puede estar vacía'; }  
  else if (value.length < 8) {  
    return 'La contraseña debe tener al menos 8 caracteres';  
  } // Añadir más condiciones según sea necesario  
  return null;  
}
```

Mostrar Mensajes de Error Personalizados

Además de los mensajes de error integrados, se puede utilizar widgets como *SnackBar* o cuadros de diálogo para mostrar errores o confirmaciones.

```
if (!_formKey.currentState.validate()) {  
  ScaffoldMessenger.of(context).showSnackBar(  
    SnackBar(content: Text('Por favor corrige los errores en el formulario')),  
  );  
}
```

Validación de Formularios Complejos

En formularios más complejos con lógica de validación avanzada, considera separar la lógica de validación en clases o funciones separadas para mantener el código limpio y mantenible.

Para validaciones más complejas, como la verificación de formatos de correo electrónico, números de teléfono, etc., puede utilizar paquetes de terceros disponibles en pub.dev³⁴.

La implementación efectiva de validaciones en los formularios Flutter es esencial para garantizar que los datos recopilados sean precisos y útiles, al proporcionar retroalimentación clara y oportuna a través de mensajes de error, puedes mejorar significativamente la experiencia del usuario y la calidad de los datos recopilados. Mantén las validaciones específicas, claras y amigables para ayudar a los usuarios a completar los formularios de manera eficiente.

3.4 Ejercicio: hacer interactiva la aplicación de lista de planes

Este ejercicio se amplía el proyecto del Capítulo 2, centrándose ahora en proporcionar la funcionalidad para añadir y eliminar planes en la aplicación de lista de planes.

Inicialmente se presenta los archivos y carpetas que se requieren para continuar con el ejercicio.

³⁴ <https://pub.dev/>

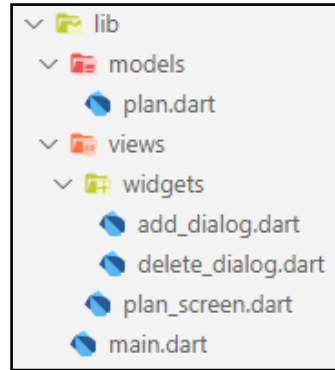


Figura III.4. Estructura del directorio lib de la aplicación

La novedad está en la creación de la carpeta “models” y los archivos “plan.dart” y “add_dialog.dart”, ya que el resto fue creado en el capítulo 2.

3.4.1 Instrucciones

En este ejercicio se trabajará inicialmente en la conversión del widget *PlanScreen*, de tipo *Stateless* a tipo *Stateful* con la finalidad de poder trabajar con un estado. Esto significa que se debe implementar un modelo Plan, también, se debe aplicar la funcionalidad de añadir y eliminar planes, por lo que se hace necesario aplicar los diálogos respectivos.

El resultado al finalizar el ejercicio será el siguiente:

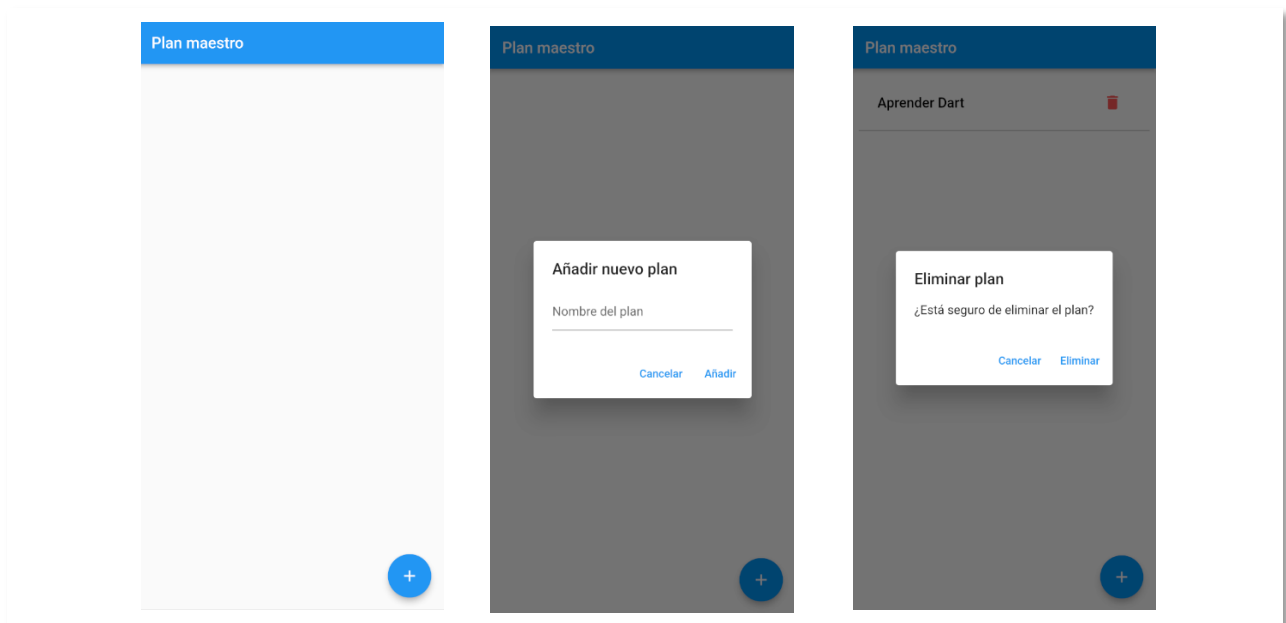


Figura III.5. Pantallas de la aplicación lista de planes

Requisitos Previos.

Para empezar con el ejercicio se debe haber transitado y tener la aplicación resultante del ejercicio de la Sección 4 de Capítulo 2.

3.4.2 Creación de un diálogo para añadir planes

Se hace notar que en el capítulo anterior se trabajó en el diálogo para la eliminación de planes. Ahora, en este punto se trabajará en el diálogo que apoyará la agregación de planes, aprovechando así el contenido que se estudió en el presente capítulo y que se hace necesario para lograr este widget. Este diálogo requiere que se introduzca un nombre para el plan, lo cual no se podría realizar sin el concurso de los widgets de entrada que ahora ya son parte de lo que se conoce. A continuación, se presenta el código para ese diálogo:

views/widgets/add_dialog.dart

```
import 'package:flutter/material.dart';
class AddDialog extends StatelessWidget {
  final TextEditingController _textController = TextEditingController();
  final String title; final String decoration; final Function(String) onAdd;
  AddDialog({
    Key? key, required this.title, required this.decoration, required this.onAdd
  }): super(key: key);
  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(title),
      content: TextField(
        controller: _textController,
        decoration: InputDecoration(hintText: decoration),
      ),
      actions: <Widget>[
        TextButton(
          onPressed: () { Navigator.of(context).pop(); },
          child: const Text("Cancelar"),
        ),
        TextButton(
          onPressed: () {
            if (_textController.text.isNotEmpty) {
              onAdd(_textController.text);
              _textController.clear();
              Navigator.of(context).pop();
            }
          },
          child: const Text("Añadir"),
        ),
      ],
    );
  }
}
```

Se podrá apreciar en el código, que se hace uso de un *AlertDialog* al igual que el dialogo de eliminación, sin embargo, a diferencia del anterior el presente requiere de un *TextField* y un *TextEditingController* para dotar al dialogo la capacidad de recibir una cadena desde el teclado.

3.4.3 Modificación de la vista PlanScreen

La principal característica de la aplicación ahora es la de gestionar una lista de planes, por esa razón, cobra importancia la creación del modelo *Plan*, el cual debe delinear la estructura de cada plan miembro de la lista, a continuación, se puede observar el código de este modelo:

```
models/plan_screen.dart

class Plan {
  final String name;
  Plan({this.name = ''});
}
```

Este modelo sencillo corresponde a una clase que ofrece la posibilidad de crear objetos de tipo *Plan* que poseen simplemente un nombre (*name*), que por defecto puede ser una cadena vacía.

Ahora, se debe trabajar en la conversión de la clase *PlanScreen* haciéndola *StatefulWidget*.

```
views/plan_screen.dart

import 'package:flutter/material.dart';
import '../models/plan.dart';
import '../widgets/add_dialog.dart';
import '../widgets/delete_dialog.dart';
class PlanScreen extends StatefulWidget {
  const PlanScreen({super.key});
  @override
  State<PlanScreen> createState() => _PlanScreenState();
}
class _PlanScreenState extends State<PlanScreen> {
  final List<Plan> _plans = [];
  ...
}
```

Ahora la clase es un *StatefulWidget*, además incluye recursos como el modelo (*Plan*) y los diálogos (*AddDialog*, *DeleteDialog*). Por último, algo muy importante, es que ahora se cuenta con una lista del tipo *Plan* nombrada como *_plans* e inicializada como lista vacía.

En relación al método *build(BuildContext context)* simplemente existe un cambio en relación al código que ya tenía implementado, que se presenta en el evento *onPressed* de *floatingActionButton*.

```
onPressed: () => _showAddPlanDialog(),
```

Lo que permitirá que al tocar el botón se invoque un método, el código de este método es el siguiente:

```
void _showAddPlanDialog() {  
  showDialog(  
    context: context,  
    builder: (BuildContext context) {  
      return AddDialog(  
        title: "Añadir nuevo plan",  
        decoration: "Nombre del plan",  
        onAdd: (String text) {  
          setState(() {  
            _plans.add(Plan(name: text));  
          });  
        },  
      );  
    },  
  );  
}
```

Aquí hay que hacer una pausa y notar que se hace uso del widget *AddDialog* que se construyó anteriormente, apoyado de un *ShowDialog*, algo también significativo es el manejo de la lista *_plans* para añadir un objeto de tipo *Plan*.

Se recordará que la lista de planes anteriormente se generaba de manera estática mediante un ciclo en el método *_buildList()*, ahora esto cambiará, ya que la lista de planes es dinámica, por lo tanto, hay algunas modificaciones que se debe realizar en el mismo método.

```
Widget _buildList() {  
  return ListView.builder(  
    itemCount: _plans.length,  
    itemBuilder: (context, index) {  
      final plan = _plans[index];  
      return Container(  
        margin: const EdgeInsets.all(8.0),  
        padding: const EdgeInsets.all(8.0),  
        decoration: const BoxDecoration(  
          border: Border(  
            bottom: BorderSide(  
              color: Colors.black26,  
              width: 1.0,  
            ),  
          ),  
        ),  
      );  
    },  
  );  
}
```

```

    ),
  ),
  child: ListTile(
    title: Text(
      plan.name,
      style: const TextStyle(fontWeight: FontWeight.w500, fontSize: 18.0),
    ),
    onTap: () {
      // Aquí iría tu código para manejar el onTap
    },
    trailing: IconButton(
      icon: const Icon(Icons.delete, color: Colors.redAccent),
      onPressed: () {
        _showDeletePlanDialog(index);
      },
    ),
  ),
);
},
);
}
}

```

Se podrá apreciar que ahora se genera una lista con el método *builder()* de *ListView*, la estructura se mantiene con la diferencia que ahora se indica la cantidad de elementos a construir (*itemCount: _plans.length*), como también se obtiene el objeto plan (*final plan = _plans[index]*) para acceder a sus propiedades, en este caso *plan.name*.

Finalmente es importante observar que el evento *onPressed* del *IconButton* ahora hace la invocación de la función *_showDeletePlanDialog(index)* pasando como parámetro el índice que ocupa el plan dentro la lista. La codificación del método es la siguiente:

```

void _showDeletePlanDialog(int index) {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return DeleteDialog(
        title: "Eliminar plan",
        content: "¿Está seguro de eliminar el plan?",
        onDelete: () {
          setState(() {
            _plans.removeAt(index);
          });
        },
      );
    },
  );
}

```

```
        });  
    },  
    );  
}
```

Esta función hace uso del widget que se construyó en el Capítulo 2, llamado *DeleteDialog*. Se debe notar que en su callback se hace una actualización del *state* eliminando el plan señalado por *index* de la lista mediante *_plans.removeAt(index)*.